

Leaderboard UML Planning Document

Rules & Validation + Quality-Testing

Expectations

What we are Creating

1. A use case diagram for the leaderboard
2. A class diagram for the leaderboard user/server view of everything
3. Will use draw.io to stay consistent with the platform core team

What we are Implementing

For this submission, we are modelling the leaderboard only (so not the matchmaking)

The leaderboard diagram will feature three main aspects:

1. View Leaderboard,
2. View My Score,
3. Submit Score (update the leaderboard).

Early Assumptions

I think it's fair to say that we assume:

1. Scores are stored server-side and accessed via network requests (similar to how the platform core's game architecture planning works).
2. Each specific score belongs to a specific user and is held through a userID and gameId
3. The top N players will return a list of game entries sorted solely by highest score (because it's a leaderboard)

Leaderboard Actors

For actors, we are thinking of 2 main actors (for now):

1. The Player (the primary actor for the leaderboard)
2. The Game Server (the secondary actor that will store and return the leaderboard data for the game)

Diagram Plans

General Pre/Post-conditions

Pre Conditions:

- A valid gameId is provided for any leaderboard requests
- For user-specific requests, the player is authenticated, and a valid userID is available
- The leaderboard has access to score storage (the server-side database)

Post Conditions:

- View Leaderboard: the system displays a ranked list (how it is ranked can vary) of top N scores or an empty message
 - Failure: Error message displaying any possible cause; failure to contact the server (timeout, request refused), checksum fail, leaderboard service failure
 - No data can be modified
- View My Score: The system displays the player's score result or an empty message
 - Failure: Error message displaying any possible cause; failure to contact the server (timeout, request refused), checksum fail, leaderboard service failure
 - No data can be modified
- Submit Score (working/success): A new score entry is stored server-wide and becomes available for all future leaderboard queries/requests/usage
- Submit Score (failure): No score is stored, and an error message is displayed

Use Case Diagram Plan

The name for the system boundary can be straightforward: Leaderboard

Use cases we can/will include within it are:

1. View Leaderboard (shows the top N players)
2. View My Score (shows the users highscore | or shows the user's scores (a numbered list of best to worst, potentially)
3. Submit Score (this is just adding a new score/scores onto the leaderboard). Changing the user score should also change MMR
4. Filter scores (high-low, all-time, monthly, by game, etc.)
5. Updating MMR (assists with matchmaking, should be proportional to score)

Class Diagram Plan

Includes Class Relationships + Dependencies

First Pass:

For classes, we could do something like LeaderboardClient/LeaderboardServer, or other things I haven't thought of.

- LeaderboardClient would be the local instance, and would carry PlayerID and current score.
- LeaderboardServer is the second actor, and would have methods to query certain data

Second Pass:

Include some contact to a game registry to get gameId values and related game information (akin to browser fingerprints, but for games only)

- LeaderboardClient
 - Associates with: database (abstracted to server)
- LeaderboardDataBase; handles requests/sending to server + validation
 - Associates with: database (abstracted to server)
 - Gets score data from database server (tied to userID); possibly tie to active users currently in server (default to current user data and users connected to server session?), and have global score data called upon user access of the global leaderboard?
- ScoreEntry (* -> (LeaderboardClient) 1)
 - Depends on: database (abstracted to server)
 - Gets score data from LeaderboardDataBase (server interface for Leaderboard system); data format taken from database server (to prevent formatting issues on grab)
- (optional) User;
 - Depends on: server (database)
 - User instance requests data on the current user, loggedIn
 - Should have a *guard* to check if the user is logged in right now to prevent resource wastage OR just perform whatever operations anyways, and keep their User instance alive for some reasonable timeframe, in case they simply disconnected and reconnected quickly.
- Aggregate (part) of: LeaderboardDataBase
 - Potentially housed as multiple instances of cached data to reference for connected / recently disconnected users (may cause synchronization issues)

Initial Methods

Methods:

Client side, we can/will incorporate:

1. sendScore(gameID, score)
2. requestTopScore(gameID, max)
 - a. May be altered to consider other filter orders, OR make it grab and have something else do the filtering, OR make multiple functions to perform different filtering (means filter calls for every call to read values)
3. requestUserTopScore(userID, gameID)
4. currentUserID
5. currentUsername

6. isLoggedIn

Network/Server side, we will incorporate:

1. setScore(userID, gameId, score)
 - a. resetScore() can have functionality abstracted into this method instead
OR
Have resetScore() send a request to have the database be wiped to a default blank state (DANGEROUS)
2. getTopScores(gameID, Max) / requestLeaderboard()
 - a. Same as client-sided requestTopScore(),
may be altered to consider other filter orders, OR make it grab and have something else do the filtering, OR make multiple functions to perform different filtering (means filter calls for every call to read values)
3. getUserHighTopScore(userID, gameId)
4. handleClient(socket) → **I GOT THIS FROM the platform core team's info - it is a server-wide network handler (from what I interpret it as) that runs when a client connects**

Mapping between use cases and methods (left to right on the table):

Use Case	Method	Methods
viewLeaderboard	requestTopScores(gameID, Max)	NetworkServer
viewMyScore	requestUserTopScore	Database
submitScore	sendScore	Session